Build Task — Graph Skeleton (feeds into Project 3) — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python — how a real codebase would actually write it).

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building the Real-world exercise's graph again — Doc07's agent as `think` / `act` nodes — split into proper files, plus one new thing: a **risky** tool that must not run until a human says yes.

Every piece was practiced in this document's exercises:

- **The state, `think`, `act`, `route`:** the Real-world exercise (Approach 1).
- **A 3-way conditional edge:** the Intermediate exercise.
- **`interrupt()` + `Command(resume=...)` + a checkpointer:** the Failure exercise, Part 2.
- **Doc07's tools and `run_tool()`:** copy Doc07's Build Task files in unchanged — `agent.py`, `docs_search.py`, `exceptions.py`, `http_client.py`, `logging_setup.py`, `tools.py`.

The one new file is `refunds.py`: a placeholder `@tool` (Doc06) called `issue_refund(order_id)` that just returns a sentence. It stands in for an action with real consequences.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

How the risky tool is caught:

- `think` looks at the model's `tool_calls`. No calls → `next_action = "finish"`. Calls → `"call_tool"`. Any call to a tool in `RISKY_TOOLS` → `"risky"`.
- `route` returns `state["next_action"]`; the edge maps `call_tool → act`, `risky → risky_action`, `finish → END`.
- `risky_action` calls `interrupt({"question": ...})`. On `Command(resume=True)` it runs the calls with the same helper as `act`; on `resume=False` it answers every call with a "Not approved" tool message — the model must get an answer for every call it made (Doc07).
- Both `act` and `risky_action` go back to `think`, so the model writes the final answer.

Then the parts the Intermediate exercises practiced:

- **No swallowed errors:** `run_tool()` already turns a tool failure into an error string; also log it with `get_logger()`, and keep it in `steps`.
- **A loud failure on a bad route:** the Edge cases exercise's named error — `route()` raises `UnhandledRouteError` for any `next_action` it doesn't handle.
- **`build_graph(checkpointer=None)`:** the caller chooses the checkpointer.
- **A real restart:** the Failure exercise's `SqliteSaver` and `graph.get_state(config)` — one script pauses and exits, a second, new process checks what's paused and resumes it.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state.py:  messages (added to), steps (added to), next_action, approved

nodes.py:
    ALL_TOOLS = AGENT_TOOLS + [issue_refund];  RISKY_TOOLS = ["issue_refund"]
    think(state):
        reply = model with ALL_TOOLS, invoked on state["messages"]
        next_action = "finish"; "call_tool" if it asked for tools;
                      "risky" if any call is in RISKY_TOOLS
        return {"messages": [reply], "next_action": next_action}
    run_calls(calls):   Doc07's inner loop -> tool messages + steps
    act(state):         run_calls(last message's tool_calls)
    risky_action(state):
        approved = interrupt({"question": ...})
        yes -> run_calls(...) and approved=True
        no  -> a "Not approved" tool message per call, approved=False
    route(state):       return state["next_action"]

graph.py:  build_graph() -- nodes, START -> think, the 3-way edge,
           act -> think, risky_action -> think, compile(MemorySaver())

test_graph.py: case 1 no risky action, case 2 pauses, case 3 resume
```

Here's almost the whole `risky_action` — fill in the "not approved" part:

```
def risky_action(state: AgentState) -> dict:
    calls = state["messages"][-1].tool_calls
    first = calls[0]
    question = f"Approve {first['name']} with {first['args']}?"
    approved = interrupt({"question": question})
    if approved:
        update = run_calls(calls)
        update["approved"] = True
        return update
    # your turn: one {"role": "tool", ...} message per call saying it was
```

```
    # not approved, then return them with "approved": False
    ...
```

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

```
exceptions.py: Doc07's file + UnhandledRouteError
nodes.py:      run_calls() also logs (info on success, warning on error)
               route() raises UnhandledRouteError if next_action is not
               one of HANDLED_ROUTES
graph.py:      build_graph(checkpointer=None) -> MemorySaver() if None
test_graph.py: 5 cases, each printing OK or WRONG
               (no risky action, pauses, approved, rejected, unknown route)
pause_demo.py:  SqliteSaver -> invoke the refund task -> print the question
resume_demo.py: a NEW process: SqliteSaver -> graph.get_state(config)
                -> if .next is set: Command(resume=True)
```

Run `python pause_demo.py`, let it exit, then `python resume_demo.py`. If the second script finishes the refund, state really survived a restart — the README's "from a fresh process" Test Case.

Difference between Basic and Intermediate: Basic builds the graph and checks the happy path in one process. Intermediate makes failures visible (logs and a named routing error), tests the rejection path, lets the caller choose the checkpointer, and proves the pause survives a real restart.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Read both depths below — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Your files (the 6 Doc07 files are copied in unchanged):

```
09_langgraph/practice/build_task/
├── agent.py              Doc07's: AGENT_TOOLS, run_tool(), run_agent()
├── docs_search.py        Doc07's
├── exceptions.py         Doc07's (+ UnhandledRouteError, Intermediate)
├── graph.py              build_graph()
├── http_client.py        Doc07's
├── logging_setup.py      Doc07's
├── nodes.py              think, act, risky_action, route
├── pause_demo.py         Intermediate: pause and save to SQLite
├── refunds.py            issue_refund -- the placeholder risky tool
├── resume_demo.py        Intermediate: resume in a new process
├── state.py              AgentState
├── test_graph.py         the Test Cases
└── tools.py              Doc07's
```

Basic Version

Approach 1 — Doc07's agent as a graph, with one human check

Story — `refunds.py`: a placeholder for an action with real consequences. **If not:** there would be nothing worth pausing for, and the `interrupt()` point would never be tested.

```
# refunds.py
from langchain_core.tools import tool


@tool
def issue_refund(order_id: str) -> str:
    """Issue a refund for one order. Use it only when the user asks for a
    refund on a specific order number."""
    # a placeholder: the real version would call the payments system
    return f"Refund issued for order {order_id}."
```

Story — `state.py`: one typed shape listing everything any node can read or write. **If not:** each node would guess which keys exist, and a typo would be a silent new key instead of an error your editor catches.

```
# state.py
import operator
from typing import Annotated, TypedDict


class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    steps: Annotated[list[dict], operator.add]
    next_action: str
    approved: bool
```

Story — `nodes.py`: the Real-world exercise's `think` and `act`, plus `risky_action` with the `interrupt()` point. `run_calls()` is Doc07's inner tool loop, shared by `act` and `risky_action`. **If not:** the risky tool would run the moment the model asked for it.

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from agent import AGENT_TOOLS, run_tool
from refunds import issue_refund
from state import AgentState

ALL_TOOLS = AGENT_TOOLS + [issue_refund]
RISKY_TOOLS = ["issue_refund"]


def think(state: AgentState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(ALL_TOOLS).invoke(state["messages"])
    next_action = "finish"
    if response.tool_calls:
        next_action = "call_tool"
    for call in response.tool_calls:
        if call["name"] in RISKY_TOOLS:
            next_action = "risky"
    return {"messages": [response], "next_action": next_action}


def run_calls(calls: list[dict]) -> dict:
    tool_messages = []
```

```
    steps = []
    for call in calls:
        result, error = run_tool(call, ALL_TOOLS)
        observation = result
        if error is not None:
            observation = f"Error: {error}"
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({
            "tool": call["name"],
            "arguments": call["args"],
            "result": result,
            "error": error,
        })
    return {"messages": tool_messages, "steps": steps}


def act(state: AgentState) -> dict:
    return run_calls(state["messages"][-1].tool_calls)


def risky_action(state: AgentState) -> dict:
    calls = state["messages"][-1].tool_calls
    first = calls[0]
    question = f"Approve {first['name']} with {first['args']}?"
    approved = interrupt({"question": question})
    if approved:
        update = run_calls(calls)
        update["approved"] = True
        return update

    tool_messages = []
    for call in calls:
        tool_messages.append({
            "role": "tool",
            "content": "Not approved by a human. Do not try again.",
            "tool_call_id": call["id"],
        })
    return {"messages": tool_messages, "approved": False}


def route(state: AgentState) -> str:
    return state["next_action"]
```

Story — `graph.py`: wires the nodes, the 3-way edge and the checkpointer, and returns the compiled graph. **If not:** every script would rebuild the wiring itself, and the copies would drift apart.

```
# graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from nodes import act, risky_action, route, think
from state import AgentState


def build_graph():
    builder = StateGraph(AgentState)
    builder.add_node("think", think)
    builder.add_node("act", act)
    builder.add_node("risky_action", risky_action)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route,
        {"call_tool": "act", "risky": "risky_action", "finish": END},
    )
    builder.add_edge("act", "think")
    builder.add_edge("risky_action", "think")
    return builder.compile(checkpointer=MemorySaver())
```

Story — `test_graph.py`: the first 3 Test Cases, in the plain `case` style of Doc04–07. **If not:** "it pauses before the refund" would be something you checked once by hand.

```
# test_graph.py
# Runs the Test Cases from the README. Run it from inside this folder:
# python test_graph.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.types import Command

from graph import build_graph


def start(task):
    return {
        "messages": [{"role": "user", "content": task}],
        "steps": [],
        "next_action": "",
        "approved": False,
    }
```

```
graph = build_graph()

config1 = {"configurable": {"thread_id": "test-1"}}
result1 = graph.invoke(start("What is 12 + 30?"), config=config1)
print("case 1 (no risky action) ->", result1["messages"][-1].content)
print("case 1 (paused?)         ->", "__interrupt__" in result1)

config2 = {"configurable": {"thread_id": "test-2"}}
task2 = start("Please issue a refund for order A100.")
paused = graph.invoke(task2, config=config2)
print("case 2 (paused?)         ->", "__interrupt__" in paused)

resumed = graph.invoke(Command(resume=True), config=config2)
print("case 3 (approved)        ->", resumed["approved"])
print("case 3 (answer)          ->", resumed["messages"][-1].content)
```

Expected output (the wording varies):

```
case 1 (no risky action) -> 12 + 30 is 42.
case 1 (paused?)         -> False
case 2 (paused?)         -> True
case 3 (approved)        -> True
case 3 (answer)          -> The refund for order A100 has been issued.
```

The calculation never pauses; the refund pauses at `interrupt()`, and `Command(resume=True)` on the same `thread_id` picks it up, runs `issue_refund`, and the model writes the final answer. A failed tool shows up in `steps` (its `error`) and in the tool message the model reads — nothing is swallowed. Still missing: the rejection path, a check on bad routes, and a resume from a new process.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — failures logged, a checked route, and OK/WRONG tests

Story — `exceptions.py` + `nodes.py`: `run_calls()` now logs every tool result and every failure, and `route()` raises the Edge cases exercise's `UnhandledRouteError` for a `next_action` it doesn't handle. **If not:** a failed tool would only be visible if you read the state, and a typo in a `next_action` value would be a confusing `KeyError` from inside LangGraph.

```
# exceptions.py — the class added at the end of Doc07's file
class UnhandledRouteError(Exception):
    """Raised when route() gets a next_action the graph doesn't handle."""
```

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from agent import AGENT_TOOLS, run_tool
from exceptions import UnhandledRouteError
from logging_setup import get_logger
from refunds import issue_refund
from state import AgentState

ALL_TOOLS = AGENT_TOOLS + [issue_refund]
RISKY_TOOLS = ["issue_refund"]
HANDLED_ROUTES = ["call_tool", "risky", "finish"]


def think(state: AgentState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(ALL_TOOLS).invoke(state["messages"])
    next_action = "finish"
    if response.tool_calls:
        next_action = "call_tool"
    for call in response.tool_calls:
        if call["name"] in RISKY_TOOLS:
            next_action = "risky"
    return {"messages": [response], "next_action": next_action}


def run_calls(calls: list[dict]) -> dict:
```

```
    logger = get_logger(__name__)
    tool_messages = []
    steps = []
    for call in calls:
        result, error = run_tool(call, ALL_TOOLS)
        observation = result
        if error is None:
            logger.info(f"{call['name']} -> {result}")
        else:
            # why: never swallowed -- it is logged here AND kept in state
            observation = f"Error: {error}"
            logger.warning(f"{call['name']} failed: {error}")
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({
            "tool": call["name"],
            "arguments": call["args"],
            "result": result,
            "error": error,
        })
    return {"messages": tool_messages, "steps": steps}


def act(state: AgentState) -> dict:
    return run_calls(state["messages"][-1].tool_calls)


def risky_action(state: AgentState) -> dict:
    calls = state["messages"][-1].tool_calls
    first = calls[0]
    question = f"Approve {first['name']} with {first['args']}?"
    approved = interrupt({"question": question})
    if approved:
        update = run_calls(calls)
        update["approved"] = True
        return update

    tool_messages = []
    for call in calls:
        tool_messages.append({
            "role": "tool",
            "content": "Not approved by a human. Do not try again.",
            "tool_call_id": call["id"],
        })
```

```
    return {"messages": tool_messages, "approved": False}


def route(state: AgentState) -> str:
    action = state["next_action"]
    if action not in HANDLED_ROUTES:
        raise UnhandledRouteError(
            f"route() got next_action={action!r}; "
            f"it only handles {HANDLED_ROUTES}"
        )
    return action
```

Story — `test_graph.py` (Intermediate): 5 cases, one function each, printing `OK` or `WRONG` — including the **rejection** path and a bad route. **If not:** "a rejected refund never runs" would be untested, and it's the most important promise a human-approval step makes.

```
# test_graph.py
# Runs the Test Cases from the README. Each case prints OK or WRONG.
# Run it from inside this folder: python test_graph.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from exceptions import UnhandledRouteError
from graph import build_graph
from nodes import route


def start(task: str) -> dict:
    return {
        "messages": [{"role": "user", "content": task}],
        "steps": [],
        "next_action": "",
        "approved": False,
    }


def config_for(thread_id: str) -> dict:
    return {"configurable": {"thread_id": thread_id}}


def tools_used(state: dict) -> list[str]:
    names = []
    for step in state["steps"]:
        names.append(step["tool"])
    return names
```

```
def test_no_risky_action(graph) -> None:
    final = graph.invoke(start("What is 12 + 30?"), config=config_for("t1"))
    result = "OK"
    if "__interrupt__" in final or "add" not in tools_used(final):
        result = "WRONG"
    print(f"case 1 (no risky action) -> {result}")


def test_risky_action_pauses(graph) -> None:
    task = start("Please issue a refund for order A100.")
    paused = graph.invoke(task, config=config_for("t2"))
    result = "OK"
    if "__interrupt__" not in paused:
        result = "WRONG"
    print(f"case 2 (pauses)          -> {result}")


def test_approved_resume(graph) -> None:
    final = graph.invoke(Command(resume=True), config=config_for("t2"))
    result = "OK"
    if not final["approved"] or "issue_refund" not in tools_used(final):
        result = "WRONG"
    print(f"case 3 (approved)        -> {result}")


def test_rejected_resume(graph) -> None:
    task = start("Please issue a refund for order A100.")
    graph.invoke(task, config=config_for("t4"))
    final = graph.invoke(Command(resume=False), config=config_for("t4"))
    result = "OK"
    if final["approved"] or "issue_refund" in tools_used(final):
        result = "WRONG"
    print(f"case 4 (rejected)        -> {result}")


def test_unexpected_route() -> None:
    state = start("anything")
    state["next_action"] = "delete_everything"
    result = "WRONG"
    try:
        route(state)
    except UnhandledRouteError:
        result = "OK"
    print(f"case 5 (unknown route)   -> {result}")
```

```
if __name__ == "__main__":
    graph = build_graph(checkpointer=MemorySaver())
    test_no_risky_action(graph)
    test_risky_action_pauses(graph)
    test_approved_resume(graph)
    test_rejected_resume(graph)
    test_unexpected_route()
```

Expected output:

```
case 1 (no risky action) -> OK
case 2 (pauses)          -> OK
case 3 (approved)        -> OK
case 4 (rejected)        -> OK
case 5 (unknown route)   -> OK
```

(With `LOG_LEVEL=WARNING` in `.env`; at `INFO` you'll also see one log line per tool call.)

Difference from Basic: failures are logged as well as kept in state, a bad `next_action` fails with a clear named error, and the tests check themselves — including the rejected refund, which must never call `issue_refund`.

Approach 2 — `build_graph(checkpointer=None)`, so the caller chooses

Story: Basic's `build_graph()` always makes its own `MemorySaver()`. A test wants a fresh one, and a real app wants one that survives a restart — the caller should choose. **If not:** the day you need a persisted checkpointer, you'd have to edit `graph.py` itself.

```
# graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from nodes import act, risky_action, route, think
from state import AgentState


def build_graph(checkpointer=None):
    # why: the caller chooses where checkpoints live -- memory for tests,
    # a SQLite file to survive a restart
    if checkpointer is None:
        checkpointer = MemorySaver()

    builder = StateGraph(AgentState)
    builder.add_node("think", think)
```

```
    builder.add_node("act", act)
    builder.add_node("risky_action", risky_action)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route,
        {"call_tool": "act", "risky": "risky_action", "finish": END},
    )
    builder.add_edge("act", "think")
    builder.add_edge("risky_action", "think")
    return builder.compile(checkpointer=checkpointer)
```

The test above already uses it: `build_graph(checkpointer=MemorySaver())`.

Approach 3 — pause in one process, resume in a new one

Story: the README's Test Case says *resume from a fresh process*. Calling `invoke()` twice in one script only proves the object in memory still has the state — the Failure exercise showed a new process with `MemorySaver` has nothing to resume. `SqliteSaver` writes every checkpoint to a file. **If not:** "it survives a restart" would be an untested assumption, and Project 3's approval step depends on it.

```
# pause_demo.py
# Run first: starts a refund task, pauses at the approval, saves to disk.
from langgraph.checkpoint.sqlite import SqliteSaver

from graph import build_graph

DB_PATH = "checkpoints.sqlite"
CONFIG = {"configurable": {"thread_id": "refund-A100"}}

with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
    graph = build_graph(checkpointer=checkpointer)
    task = {
        "messages": [
            {"role": "user", "content": "Please issue a refund for order A100."}
        ],
        "steps": [],
        "next_action": "",
        "approved": False,
    }
    paused = graph.invoke(task, config=CONFIG)
    question = paused["__interrupt__"][0].value["question"]
    print(f"Paused, asking: {question}")
    print("Saved to checkpoints.sqlite. Now run: python resume_demo.py")
```

```
# resume_demo.py
# Run second, as a NEW process: loads the paused run from disk, approves it.
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command

from graph import build_graph

DB_PATH = "checkpoints.sqlite"
CONFIG = {"configurable": {"thread_id": "refund-A100"}}

with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
    graph = build_graph(checkpointer=checkpointer)
    snapshot = graph.get_state(CONFIG)
    if not snapshot.next:
        print("Nothing is paused for this thread. Run pause_demo.py first.")
    else:
        print(f"Found a paused run, waiting at: {snapshot.next}")
        final = graph.invoke(Command(resume=True), config=CONFIG)
        print(f"approved: {final['approved']}")
        print(f"answer:   {final['messages'][-1].content}")
```

Run them one after the other:

```
python pause_demo.py
python resume_demo.py
```

Expected output (the second command's wording varies):

```
Paused, asking: Approve issue_refund with {'order_id': 'A100'}?
Saved to checkpoints.sqlite. Now run: python resume_demo.py
```

```
Found a paused run, waiting at: ('risky_action',)
approved: True
answer:   The refund for order A100 has been issued.
```

`resume_demo.py` never saw the first run: it opens the same file and builds the same graph with `build_graph(checkpointer=...)`. `graph.get_state(CONFIG)` reads the saved checkpoint for that `thread_id`; its `.next` names the node waiting to run (empty when nothing is paused — run `resume_demo.py` a second time and it says so). `Command(resume=True)` then continues from the saved pause. Nothing in `graph.py`, `nodes.py` or `state.py` changed — Approach 2 is what makes this a one-line swap.

Difference from Approach 1, and between Approaches 2/3: Approach 1 makes failures visible and tests every path, in one process. Approach 2 lets the caller pick the checkpointer. Approach 3 uses that to prove the pause survives a real restart.

Which one should you actually use?

All three, together — that's what Project 3 copies. Approach 1's checked route and rejection test are what make a human-approval step trustworthy; Approach 2's `build_graph(checkpointer)` and Approach 3's `SqliteSaver` pause/resume scripts are exactly Project 3's Step 2 (`pause_demo.py` / `resume_demo.py`).